YaLlega S.A. — Diagrama de clases

Nucleo de la plataforma de pedidos y entregas — arquitectura de 4 capas

Facultad de Tecnologia Informatica · Ingenieria en Sistemas · Practica de parcial

Patrones usados: **Strategy** (costo de envio) · **Proxy** (cache de las ultimas 5 cartas) · **Chain of Responsibility** (revision por plan) · **Template Method** (reportes genericos)

Modelo (YaLlegaNucleo.Modelo)

El modelo del negocio y los patrones Strategy, Proxy y Chain of Responsibility

Entidades del negocio

Cliente

- Nombre : string
- Plan : Plan
+ Habilitado : bool

Plan

+ Clasico : Plan (estatico)
+ Plus : Plan (estatico)
+ Premium : Plan (estatico)
- Nombre : string
- Nivel : int

NivelDeDemanda
«enum»

Baja
Media
Alta

Articulo

- Codigo : int
- Detalle : string
- Importe : decimal
- NivelNecesario : int
+ PlanNecesario : string

Carta

- Numero : int
- Titulo : string
- articulos : List<Articulo>
+ CuantosArticulos : int

+ Sumar(articulo) : Carta

CatalogoDeCartas

- cartas : List<Carta>

+ Sumar(carta)
+ DameCarta(numero) : Carta
+ DameArticulo(codigo) : Articulo
+ TodosLosArticulos() : IReadOnlyList<Articulo>

Orden

- Numero : int
- Cliente : Cliente
- Envio : CalculadoraDeEnvio
- Abierta : bool
- lineas : List<LineaDeOrden>

+ Sumar(articulo, unidades)
+ Sacar(articulo)
+ ImporteDeArticulos() : decimal
+ ImporteTotal() : decimal
+ Cerrar()
+ Tiene(articulo) : bool

LineaDeOrden

+ Articulo : Articulo
+ Unidades : int
+ Importe : decimal

CalculadoraDeEnvio

- modalidad : IModalidadDeCobro
- aplicadas : List<string>
+ TarifaBase : decimal
+ Demanda : NivelDeDemanda
+ ModalidadActual : string

+ Cobrar() : decimal
+ CambiarDemanda(demanda)
+ CambiarModalidad(modalidad)

OrdenRegistrada

+ Fecha : DateTime
+ Cliente : string
+ Articulos : string
+ TarifasDeEnvio : List<string>
+ CartaServidaDesde : string
+ IntentosRechazados : int

Carpeta Excepciones

AccesoNoPermitidoException

(sin miembros)

ArticuloDesconocidoException

+ Codigo : int

ArticuloNoEstaEnLaOrdenExceptio

+ Detalle : string

CantidadFueraDeRangoException

+ Unidades : int

OrdenYaCerradaException

+ Numero : int

Modelo / Cobros (Strategy)

Formas de cobrar el envio, intercambiables mientras la orden sigue abierta

Modalidades de cobro

IModalidadDeCobro
«interface»

+ Nombre : string
+ Calcular(tarifaBase, demanda) : decimal

CobroPorDemanda

- RecargoParaDemandaMedia : decimal
- RecargoParaDemandaAlta : decimal

+ Calcular(tarifaBase, demanda) : decimal

CobroPromocional

+ Calcular(tarifaBase, demanda) : decimal

Modelo / Cartas (Proxy)

Las cartas viven en un servidor remoto; el cache guarda las ultimas 5 en el dispositivo

Origen de las cartas y cache

IOrigenDeCartas
«interface»

+ DameCarta(numero) : Carta
+ UltimoOrigen : string

RepositorioRemotoDeCartas

- catalogo : CatalogoDeCartas
+ Consultas : int

+ DameCarta(numero) : Carta

CacheDeCartas

- servidor : IOrigenDeCartas
- cuantasGuarda : int
- guardadas : Dictionary<int, Carta>
- ordenDeUso : List<int>
+ VecesQueSirvioElCache : int
+ CartasGuardadas : int

+ DameCarta(numero) : Carta

Modelo / Revision (Chain of Responsibility)

Cadena de revisiones que impide pedir articulos fuera del plan del cliente

Revisores

RevisionDeOrden

+ Cliente : Cliente
+ CodigoDeArticulo : int
+ Articulo : Articulo

IRevisorDeOrden
«interface»

+ Revisar(revision)
+ Enganchar(siguiente) : IRevisorDeOrden

RevisorDeOrden
«abstracta»

- siguiente : IRevisorDeOrden

+ Revisar(revision)
+ Enganchar(siguiente)
Controlar(revision) *abstracto*

RevisorDeArticulo

- catalogo : CatalogoDeCartas

Controlar(revision)

RevisorDeCliente

Controlar(revision)

RevisorDePlan

Controlar(revision)

Datos (YaLlegaNucleo.Datos)

Guardado de las ordenes y escritura de los listados (Template Method)

Guardado de las ordenes

IArchivoDeOrdenes
«interface»

+ Carpeta : string
+ Ruta : string
+ Guardadas : IReadOnlyList<OrdenRegistrada>
+ Guardar(orden)
+ LeerLineas() : IReadOnlyList<string>

ArchivoDeOrdenes

- texto : TextoDeOrden

+ Guardar(orden)
+ LeerLineas()

TextoDeOrden

+ Armar(orden) : string

ErrorDeArchivoException

+ Ruta : string

Reportes

ReporteDeListado<T>
«abstracta»

+ Escribir(lista, rutaArchivo)
Encabezado() : string *abstracto*
Renglon(dato) : string *abstracto*

ReporteAutomatico<T>

- Separador : string

Encabezado() : string
Renglon(dato) : string

ReporteDeOrdenes

Encabezado() : string
Renglon(dato) : string

Negocio (YaLlegaNucleo.Negocio)

Las reglas del negocio y el armado de todo el nucleo

Servicios y armado

ServicioDeOrdenes

- primerRevisor : IRevisorDeOrden
- cartas : IOrigenDeCartas
- archivo : IArchivoDeOrdenes
+ TarifaBaseDeEnvio : decimal
+ IntentosRechazados : int
+ CartaServidaDesde : string

+ PedirCarta(numero) : Carta
+ NuevaOrden(numero, cliente, demanda)
+ Sumar(codigo, unidades)
+ Sacar(codigo)
+ AvisarDemanda(demanda)
+ PonerPromocion()
+ SacarPromocion()
+ CerrarYGuardar() : OrdenRegistrada

ServicioDeReportes

- carpeta : string

+ Escribir<T>(lista, nombreArchivo) : string
+ EscribirOrdenes(lista) : string

Nucleo

+ Ordenes : ServicioDeOrdenes
+ Reportes : ServicioDeReportes
+ ConsultasAlServidor : int
+ VecesQueSirvioElCache : int
+ CartasGuardadas : int

ArmadorDelSistema

+ Armar(catalogo, carpetaOrdenes, carpetaReportes, cuantasGuardar) : Nucleo

Consola (YaLlegaNucleo.Consola)

El ejemplo, las verificaciones y el menu

Presentacion

Programa

- Main(args) : int
- Ejemplo()
- MostrarMenuDeOpciones()
- MostrarOrden()

Verificaciones

+ Correr()
- Comprobar(queSeProbo, salioBien)
- Rechaza<T>(queSeProbo, accion)

Ejemplos

+ ArmarCatalogo() : CatalogoDeCartas
+ ArmarClientes() : List<Cliente>

Relaciones entre las clases

Origen	Destino	Relacion
CalculadoraDeEnvio	IModalidadDeCobro	guarda una modalidad de cobro (Strategy)
CobroPorDemanda	IModalidadDeCobro	implementa (tarifa base, +25 % o +60 % segun la demanda)
CobroPromocional	IModalidadDeCobro	implementa (no se cobra el envio)
Orden	CalculadoraDeEnvio	tiene una, aparte de los articulos
Orden	LineaDeOrden	tiene una lista
LineaDeOrden	Articulo	apunta a uno
CatalogoDeCartas	Carta	tiene una lista
Carta	Articulo	tiene una lista
ServicioDeOrdenes	OrdenRegistrada	la arma cuando cierra la orden
CacheDeCartas	IOrigenDeCartas	implementa (Proxy)
RepositorioRemotoDeCartas	IOrigenDeCartas	implementa (sujeto real)
CacheDeCartas	RepositorioRemotoDeCartas	lo envuelve y le agrega el cache
CacheDeCartas	Carta	guarda las ultimas 5 que se pidieron
RepositorioRemotoDeCartas	CatalogoDeCartas	busca las cartas en el catalogo
RevisorDeArticulo	RevisorDeOrden	hereda
RevisorDeCliente	RevisorDeOrden	hereda
RevisorDePlan	RevisorDeOrden	hereda
RevisorDeOrden	IRevisorDeOrden	implementa (Chain of Responsibility)
RevisorDeOrden	RevisionDeOrden	la revisa y se la pasa al siguiente
RevisorDeArticulo	CatalogoDeCartas	busca el articulo para completar la revision
ArchivoDeOrdenes	IArchivoDeOrdenes	implementa
ArchivoDeOrdenes	TextoDeOrden	la usa para armar cada linea
ReporteAutomatico<T>	ReporteDeListado<T>	hereda y resuelve los pasos con reflexion
ReporteDeOrdenes	ReporteDeListado<T>	hereda y usa sus propias columnas
ServicioDeOrdenes	IRevisorDeOrden	arranca la cadena de revision
ServicioDeOrdenes	IOrigenDeCartas	le pide las cartas (sin saber de donde salen)
ServicioDeOrdenes	IArchivoDeOrdenes	guarda las ordenes
ArmadorDelSistema	Nucleo	lo arma con todo conectado

Como leer el diagrama. Las flechas estan detalladas en la tabla de arriba. Las dependencias van en un solo sentido: Consola → Negocio → Datos → Modelo.

Strategy. El costo de envio no lo calcula `Orden`: se lo pregunta a una `CalculadoraDeEnvio`, que tiene guardada una `IModalidadDeCobro`. Hay dos: `CobroPorDemanda` (tarifa base, +25 % o +60 % segun la demanda) y `CobroPromocional` (no se cobra). Como la modalidad es un objeto aparte, se puede cambiar mientras la orden sigue abierta.

Proxy. La consola siempre le pide las cartas a `IOrigenDeCartas`. `RepositorioRemotoDeCartas` es el servidor de verdad y `CacheDeCartas` lo envuelve: primero mira lo que tiene guardado y, si no esta, se lo pide al servidor. Como tiene la misma interfaz, la consola **no distingue** de donde salio la carta. El cache guarda 5 como maximo y va descartando la mas vieja.

Chain of Responsibility. `RevisorDeOrden` guarda el revisor siguiente y le pasa la `RevisionDeOrden`. Cada revisor escribe solamente su control y tira excepcion si algo no esta permitido. Los tres son: el articulo existe, el cliente esta habilitado y el plan alcanza. Para agregar otro control se agrega un revisor y se engancha, sin tocar los demas.

Template Method. `ReporteDeListado<T>.Escribir()` tiene **la unica rutina**: encabezado, renglones y pie. Lo unico que cambia son `Encabezado()` y `Renglon()`. `ReporteAutomatico<T>` los resuelve con reflexion, asi que sirve para cualquier clase, incluso las que se agreguen despues. `ReporteDeOrdenes` es el ejemplo de columnas propias.